

Prueba Entrada Mercadona

Java Spring Boot

Autor	Javier Mercé, Salvador Pérez
Versión	2.0
Fecha	25/03/2025

Índice

<i>Sumario y Estructura</i>	2
<i>Ejercicio</i>	2
Iteración 1 (desarrollo básico)	2
Iteración 2 (reportes)	3
Iteración 3 (integración)	3
Iteración 4 (ampliación modelo)	4
<i>Funcionalidades Adicionales</i>	5

Sumario y Estructura

Prueba destinada a evaluar los conocimientos prácticos del candidato. La prueba se compone de un desarrollo básico, sobre el que se irán solicitando mejoras. Se pueden abordar las mejoras en el orden que se prefiera, aunque hay que tener en cuenta que la mejora 3 depende de la 2. El resto son independientes entre sí y solo requieren de haber completado el ejercicio base.

No es necesario realizar todas las partes de la prueba, pero cuanto más se avance más información tendrá el corrector para evaluar los conocimientos del candidato. ***Para poder evaluar la prueba, el desarrollo básico ha de poderse ejecutar y probar.*** Los pasos necesarios para probar la aplicación serán descritos por el aspirante en **un fichero README adjunto en la solución**. Se espera una **colección para las llamadas implementadas en la API**, ya sea postman, curl, Insomnia, etc. Especificar en el README donde está esa colección y que herramienta se debe utilizar.

Ejercicio

Una importante cadena de supermercados necesita una aplicación para gestionar sus tiendas, y ha contactado con nosotros para que desarrollemos una solución que les permita llevar un registro de las secciones de sus tiendas y de sus trabajadores.

Iteración 1 (desarrollo básico)

Tras reunirnos con los stakeholders de la empresa, hemos podido averiguar lo siguiente sobre cómo está organizado el negocio:

La cadena tiene tiendas abiertas por toda España. Les interesa registrar el código de tienda (único) y nombre.

Sus tiendas están divididas en secciones. Necesitan que sus secciones tengan nombre y número de horas de trabajo diarias necesarias para que esa sección esté bien atendida.

Aunque quieren que estas secciones se puedan modificar en un futuro, de momento aceptan que sean una lista predefinida.

Las secciones son Horno, Cajas, Pescadería, Verduras y Droguería. Todas las secciones necesitan 16 horas de trabajo diarias, menos Horno que necesita 8.

Las tiendas tienen trabajadores, de los cuales se guardan nombre, apellidos y DNI.

Los trabajadores están asignados a secciones. Un trabajador tiene un máximo de 8 horas de trabajo disponibles (dependiendo de su contrato), que se tienen que distribuir entre una o varias secciones de la tienda. Por ejemplo, un trabajador podría estar 2 horas en pescadería y 6 horas en horno.

Se pide:

- Crear un modelo lógico que represente lo más fielmente posible el negocio de la cadena de supermercados.
- Crear una API con las operaciones de consulta, creación, edición y borrado de trabajadores de una tienda. Incluyendo la tienda a la que están asignados y las horas disponibles según su contrato.
- Crear una API para poder asignar/desasignar a un trabajador a una sección durante cierto número de horas. Asegurándonos de que no excedemos las horas disponibles del trabajador.

Iteración 2 (reportes)

Desde dirección de la cadena de supermercados se ha pedido poder generar algunos informes para poder contrastar mejor cómo van las cosas en cada tienda. A nuestro equipo se le ha asignado la tarea de preparar la API para devolver los datos de estos informes. Se necesita:

- Un informe con el estado de una tienda. Se tiene que mostrar el nombre de la tienda, sus secciones y los trabajadores que están asignados a cada sección con el número de horas que cada trabajador va a pasar ahí.
- Otro informe con el nombre de la tienda junto con aquellas secciones de que no tienen cubiertas todas sus horas, y cuantas horas les faltan por cubrir.

Iteración 3 (integración)

Otro equipo de la importante cadena de supermercados ha creado una API para recuperar las tiendas con su nombre y dirección. Han creado una pequeña aplicación disponible como una imagen en DockerHub. Tenemos que levantar un

contenedor con esa imagen en local, sabiendo que la aplicación arranca conectada al puerto 8080.

Cuando generemos los reportes de la parte 2 deberemos consultar la API de tiendas, e incorporar la dirección de la tienda a los datos de los 2 informes.

La imagen es [jamerall/stores:latest](http://jamerall.com/stores/latest)

Se puede consultar el contrato en la ruta /docs

Iteración 4 (ampliación modelo)

La cadena mantiene una lista de aptitudes con su descripción (por ejemplo, Limpiar pescado u Hornear Pan). Aunque quieren que esta lista de aptitudes sea dinámica, y pueda cambiar con el tiempo, de momento se conforman con que se den de alta las que utilizan hoy en día.

Las secciones tienen una lista de las aptitudes que deberían tener los trabajadores que operen en ellas. Estas aptitudes son:

SECCIÓN	APTITUDES
HORNO	Hornear Pan - Repostería
CAJAS	Simpatía - Matemáticas
PESCADERÍA	Manejo de armas blancas – Limpiar pescado
VERDURAS	Fortaleza física
DROGUERÍA	Alquimia

Los trabajadores tienen una lista de las aptitudes que han adquirido. La lista de aptitudes es la misma para trabajadores y secciones.

Modificar el modelo lógico de la aplicación para reflejar la relación de los trabajadores y las secciones con las aptitudes.

Crear una operación en la API que dado un código de tienda devuelva los datos de la tienda, sus secciones y las aptitudes necesarias de estas secciones.

Funcionalidades Adicionales

Se valorará cualquier extra que el candidato incluya durante la resolución del ejercicio. Como puede ser:

- Gestión migraciones BBDD
- Contrato OpenAPI/Swagger
- Internacionalización
- Autenticación
- Uso de JPQL sobre queries nativas

Esto son solo ejemplos, cualquier extra que mejore la calidad técnica de la prueba será valorado.

Apunte importante: El código entregado tiene que funcionar y se debe disponer de datos para poder probar la funcionalidad. Como importar esos datos se deja a criterio del candidato, sabiendo que puede hacer uso del fichero README para explicar ese proceso o el juego de datos que se debería utilizar.